



Learning Representations on Temporal Graphs

How models capture evolutionary node patterns
and predict future links?

2025-12-04

SuYong Jeong

MINDS Lab

Outline



- ☐ Introduction

- ☐ JODIE

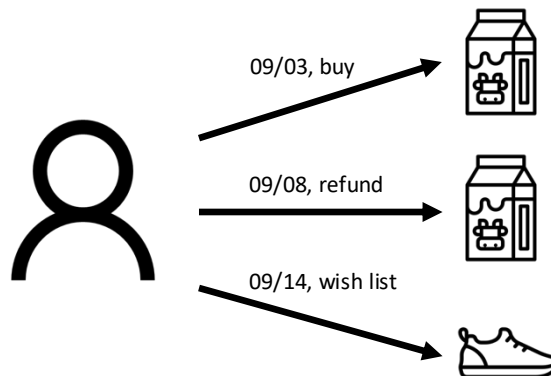
- ☐ TGAT

- ☐ TGN

- ☐ GraphMixer

□ Interactions in Real-World

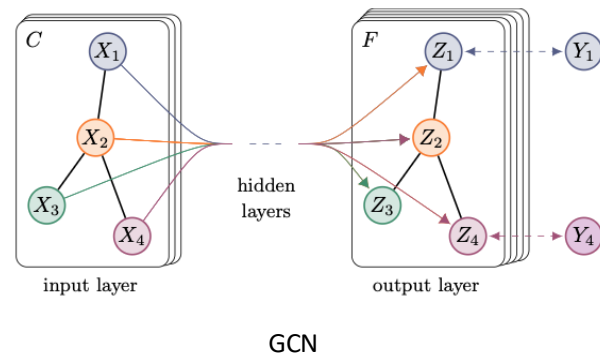
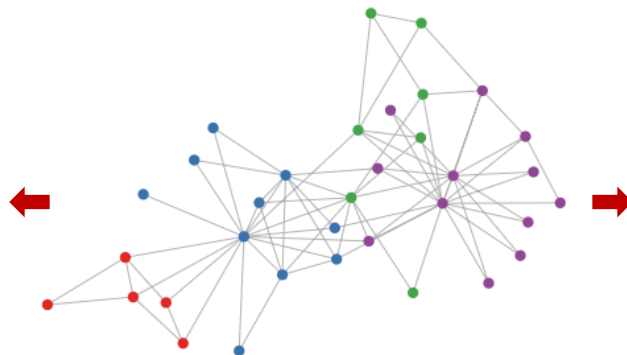
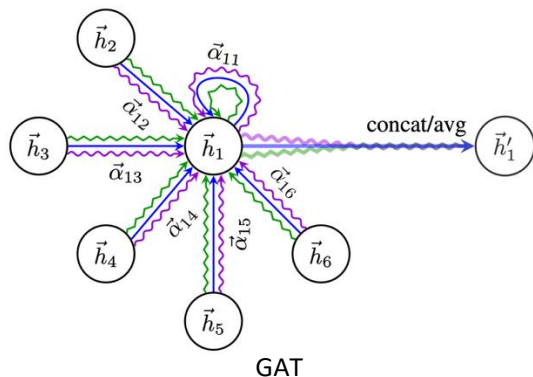
- Sequentially with various items
 - Registering lectures, purchasing items
- Dynamic, evolving over time, not static
 - Our needs and intents are always changing
- Graphs are generated by nodes interacting in a timely manner



Introduction

Existing GNNs

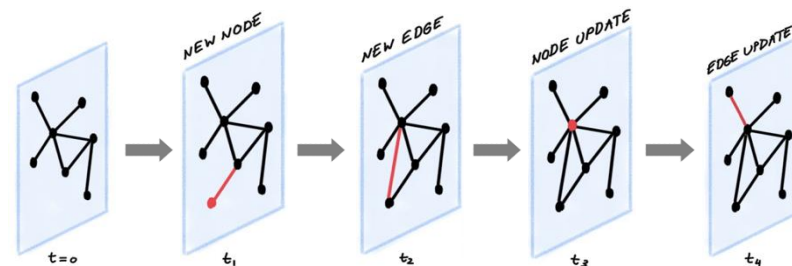
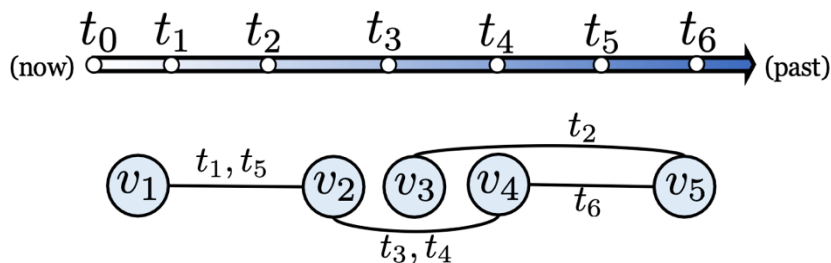
- Assume the underlying graph is static
- Insufficient to represent real-world characteristic
 - Only using the final state as a static snapshot of the graph
 - Disable to reflect evolving graph structure



Introduction

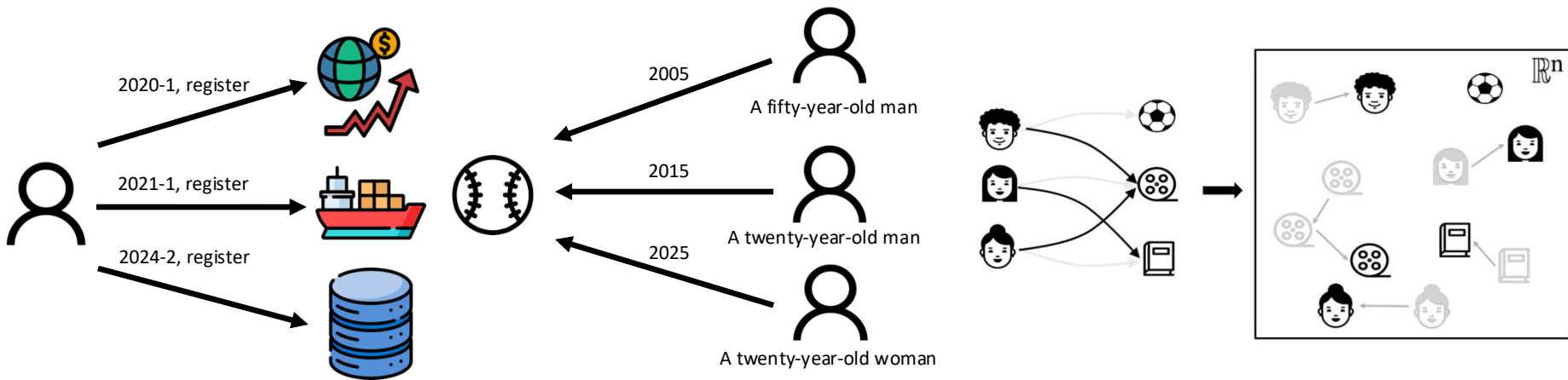
□ Temporal Graphs

- Including temporal features in each link
 - (v_1, v_2, t, f)
 - Per-link timestamps indicate when two nodes interact
- Interactions between entities are represented by links
- Representing time-evolving and dynamic interactions



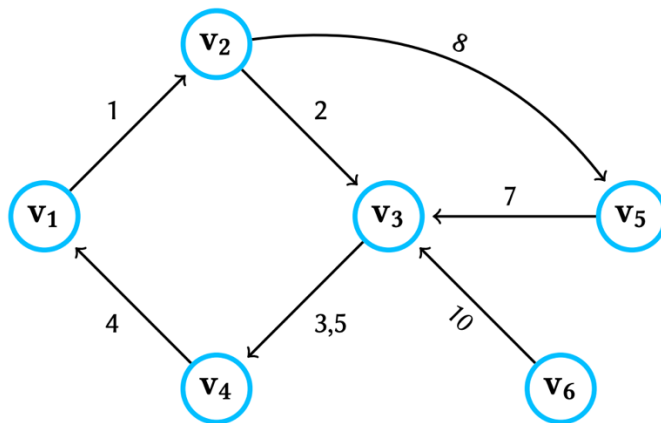
□ Temporal Node Embeddings

- Insufficient to represent the changing behavior of the entities with a static embedding
- Embeddings must continuously reflect graph's changes
- Continuously reflecting changes using interaction information



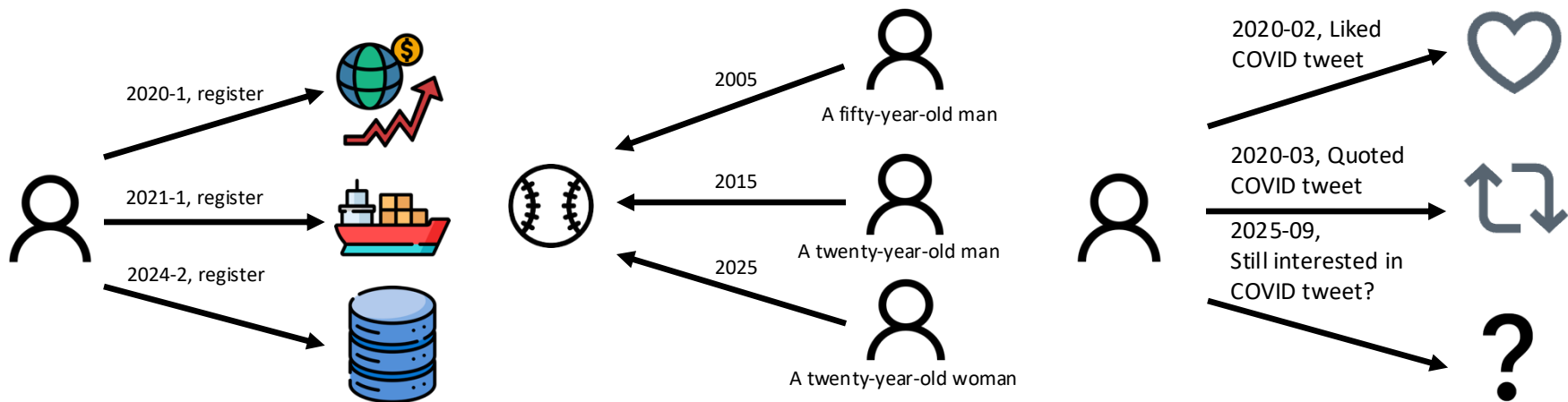
□ An Existing Temporal Node Embedding Method

- CTDNE (Continuous-Time Dynamic Network Embeddings) based on temporal random walk
- Only Generating embeddings for the final state of temporal graph
 - No direct parameterization of temporal information in the embedding
- Unable to be directly applied to the inductive setting



□ Some Challenges in Temporal Node Embeddings

- How to incorporate the temporal features of interactions?
- Using same weights to every interaction?
- How to handle historical information?
- How to handle nodes with no interactions or long-inactive nodes





Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks

Srijan Kumar^{1, 3}, Xikun Zhang², Jure Leskovec¹

1. Stanford University 2. University of Illinois, Urbana-Champaign 3. Georgia Institute of Technology

KDD 2019

JOint Dynamic User-Item Embeddings (JODIE)

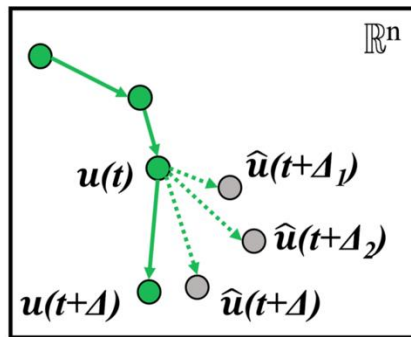
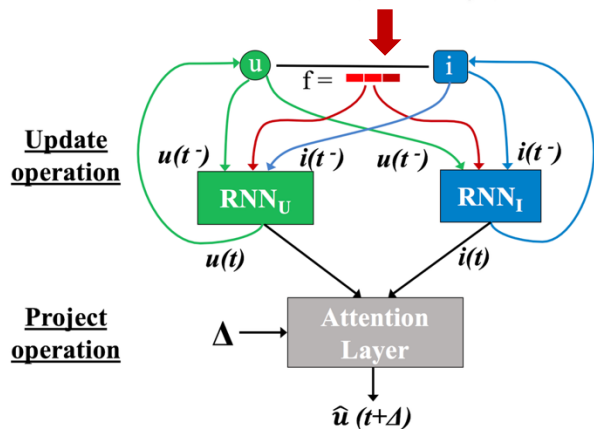
Update operation

- A dynamic co-evolution model with two RNNs
- Shared across all users and among all items

Project operation

- Forecasting the user embedding trajectories at any time

$$S = (u, i, t, f)$$



Symbol	Meaning
$u(t)$ and $i(t)$	Dynamic embedding of user u and item i at time t
$u(t^-)$ and $i(t^-)$	Dynamic embedding of user u and item i before time t
$\bar{u}$ and $\bar{i}$	Static embedding of user u and item i
$\hat{u}(t)$	Projected embedding of user u at time t
$\hat{j}(t)$	Predicted item j embedding

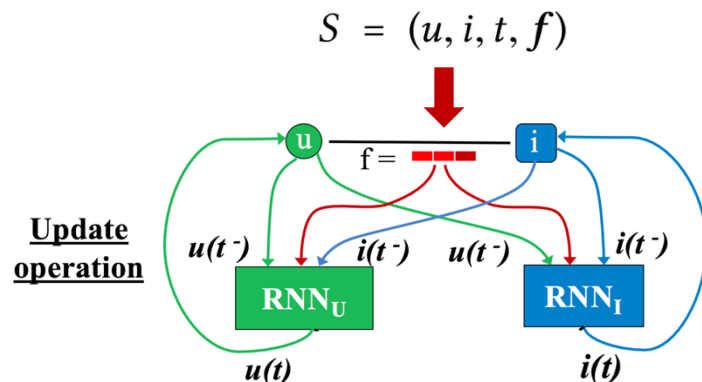
Legend:

- Update operation (solid green arrow)
- Project operation (dashed green arrow)
- User embedding u (green circle)
- Projected user embedding $\hat{u}$ (grey circle)

elapsed time $\Delta_1 < \Delta_2 < \Delta$

□ Update Operation

- Mutually-recursive RNNs
- The hidden states of the user and item RNNs represent their respective embeddings

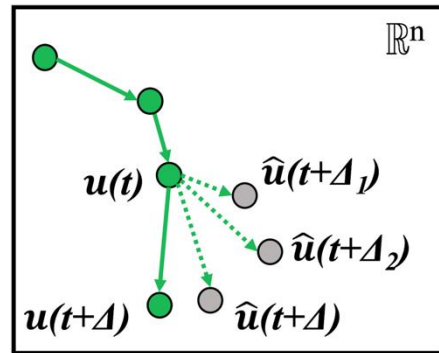
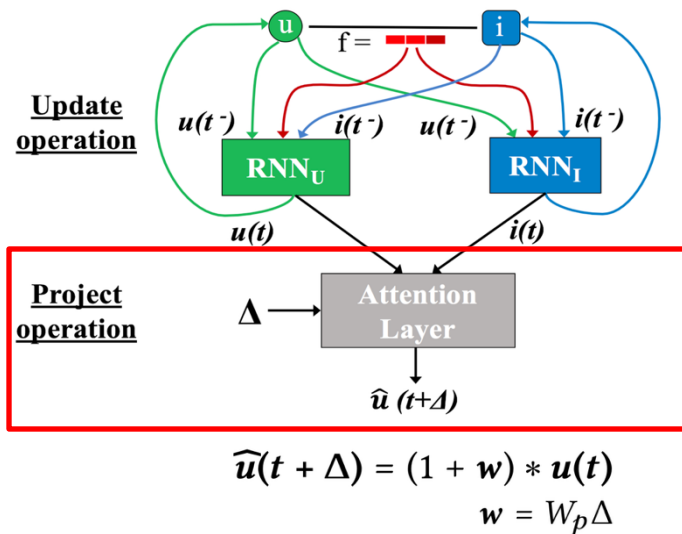


$$u(t) = \sigma(W_1^u u(t^-) + W_2^u i(t^-) + W_3^u f + W_4^u \Delta_u)$$

$$i(t) = \sigma(W_1^i i(t^-) + W_2^i u(t^-) + W_3^i f + W_4^i \Delta_i)$$

□ Project Operations

- Predicting future embedding trajectory of the user
- The drift of the projected embedding vector varies with Δ

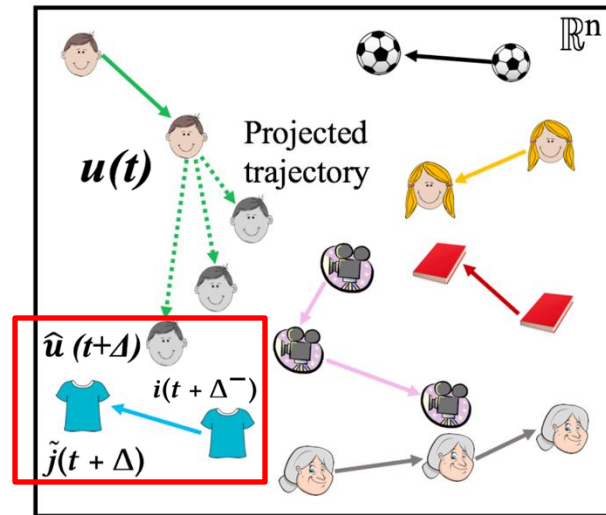


□ Predicting Next Item Embedding

- No prediction of an interaction probability between user u and item j
- Directly output an item embedding vector $\tilde{j}(t + \Delta)$ using $\mathbf{i}(t + \Delta^-)$
 - The embedding contains more recent information
 - Users often interact with the same item consecutively

$$\tilde{j}(t + \Delta) = W_1 \widehat{\mathbf{u}}(t + \Delta) + W_2 \bar{\mathbf{u}} + \underline{W_3 \mathbf{i}(t + \Delta^-)} + W_4 \bar{\mathbf{i}} + B$$

$$\begin{aligned} Loss = & \sum_{(u, i, t, f) \in S} \|\tilde{j}(t) - [\bar{i}, i(t^-)]\|_2 \\ & + \lambda_U \|\mathbf{u}(t) - \mathbf{u}(t^-)\|_2 + \lambda_I \|\mathbf{i}(t) - \mathbf{i}(t^-)\|_2 \end{aligned}$$



□ Future Interaction Prediction Experiment

- Using mean reciprocal rank (MRR) and recall@10
- JODIE outperforms all baselines by at least 20% in MRR and 14% in recall@10

Method	Reddit		Wikipedia		LastFM		Minimum % improvement of <i>JODIE</i> over method	
	MRR	Recall@10	MRR	Recall@10	MRR	Recall@10	MRR	Recall@10
LSTM [52]	0.355	0.551	0.329	0.455	0.062	0.119	104.5%	54.6%
Time-LSTM [52]	0.387	0.573	0.247	0.342	0.068	0.137	87.6%	48.7%
RRN [45]	0.603	0.747	0.522	0.617	0.089	0.182	20.4%	14.1%
LatentCross [8]	0.421	0.588	0.424	0.481	0.148	0.227	31.8%	35.2%
CTDNE [33]	0.165	0.257	0.035	0.056	0.01	0.01	340.0%	231.5%
DeepCoevolve [11]	0.171	0.275	0.515	0.563	0.019	0.039	44.8%	46.0%
<i>JODIE</i> (proposed)	0.726	0.852	0.746	0.822	0.195	0.307	-	-

Experiment

☐ User State Change Prediction

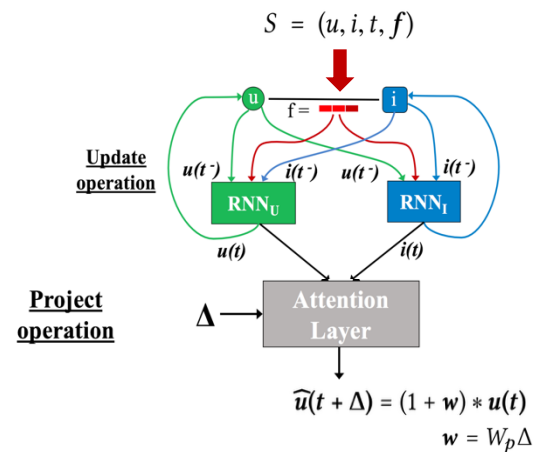
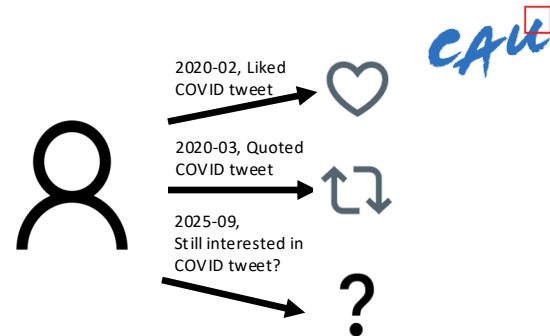
■ Using AUC

■ JODIE outperforms the baselines by at least 12.63% on average

Method	Reddit	Wikipedia	MOOC	Mean improvement of <i>JODIE</i>
LSTM	0.523	0.575	0.686	23.08%
Time-LSTM	0.556	0.671	0.711	12.63%
RRN	0.586	0.804	0.558	13.69%
LatentCross	0.574	0.628	0.686	15.62%
DeepCoevolve	0.577	0.663	0.671	13.94%
<i>JODIE</i>	0.599	0.831	0.756	-

Conclusion

- Updating Only When Action Occur
 - Missing changes in user intents
- A Dynamic Evolution Model with Two RNNs
- Training Project Operation
 - Forecasting the user embedding trajectories at any time
- Directly Output an Item Embedding Vector
 - Forecasting the item a user will interact with



$$\tilde{j}(t + \Delta) = W_1 \hat{u}(t + \Delta) + W_2 \bar{u} + W_3 i(t + \Delta^-) + W_4 \bar{i} + B$$



Inductive Representation Learning on Temporal Graph

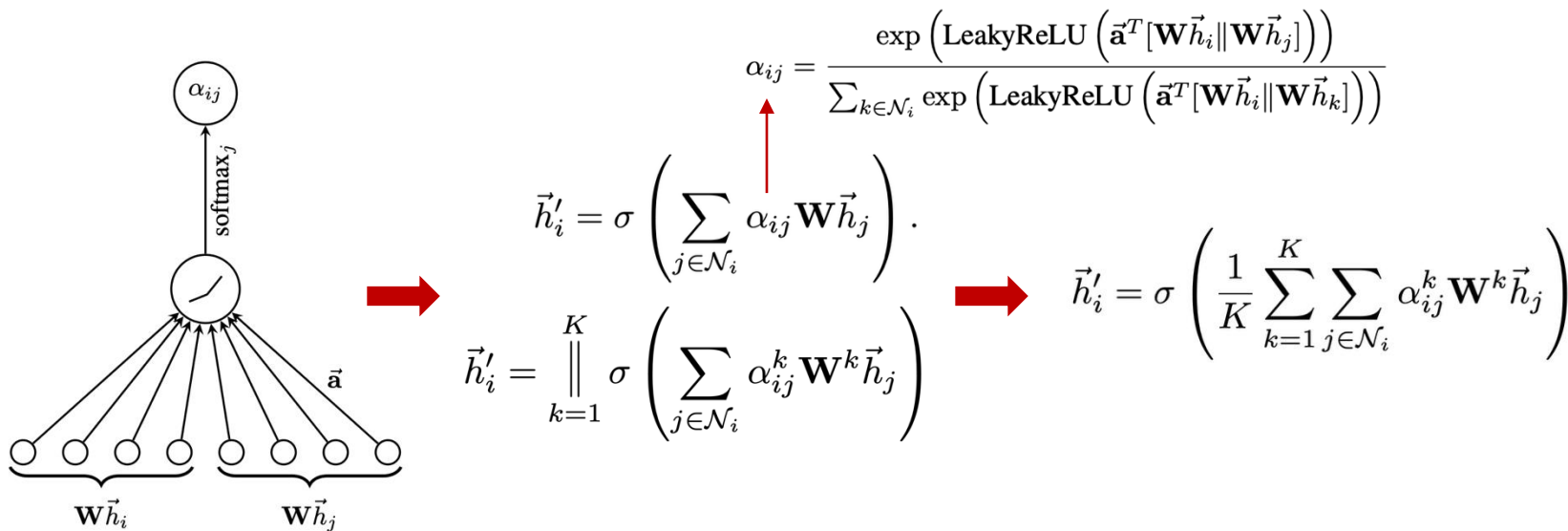
Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, Kannan Achan

Walmart Labs

ICLR 2020

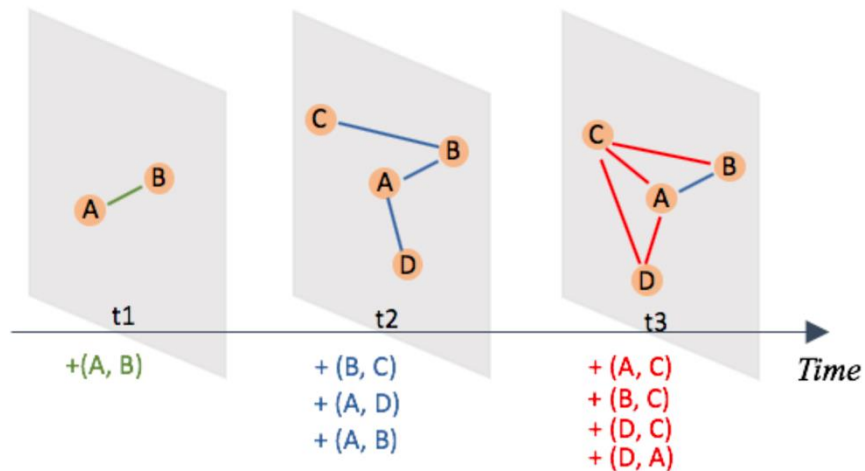
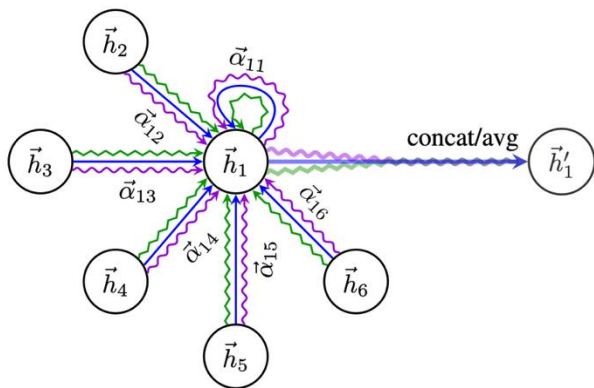
□ Brief Review of GAT

- Applicable to inductive problems with scalability



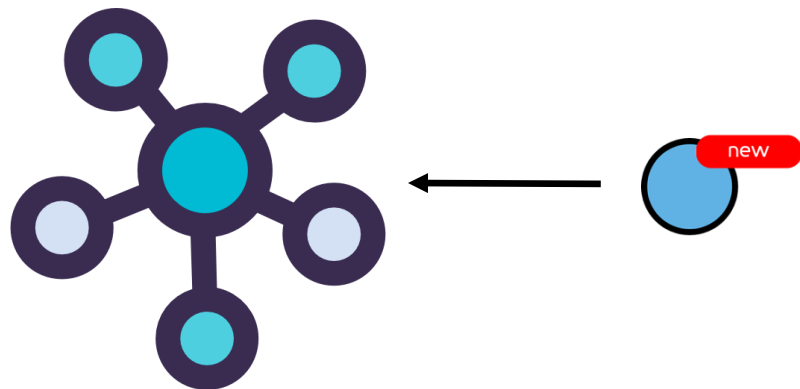
□ Temporal Graph Attention Network (TGAT)

- Inspired from GAT, inductive model
- Generating time-aware embeddings by learning temporal representations
- Better representations of dynamic real-world characteristics
 - Evolving node features and topological structures



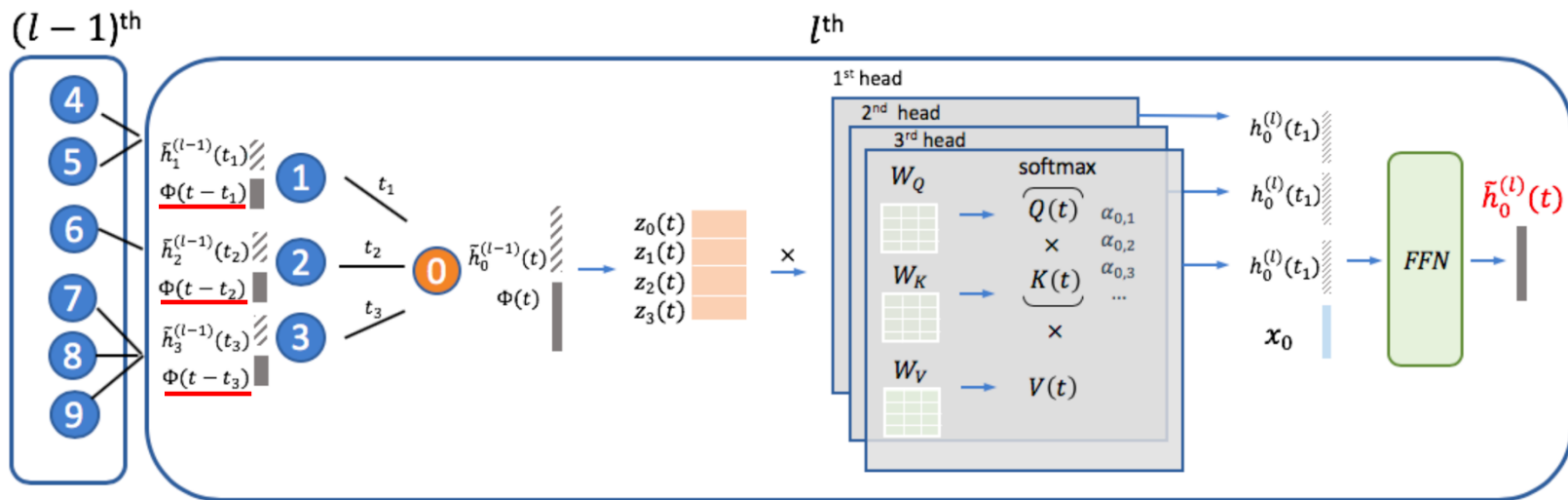
☐ Why Is the Inductive Setting Important?

- Unseen nodes can emerge at any time
 - ☐ Especially in dynamic graphs
- Modeling unseen nodes during training without retraining
- Ensuring scalability for large-scale graphs with new nodes



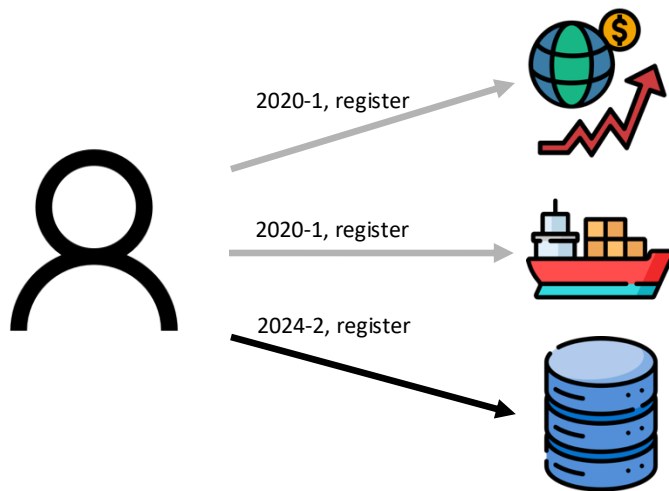
□ An Overall Structure of TGAT

- GAT + Temporal feature Encoding + Multi head self-attention Mechanism



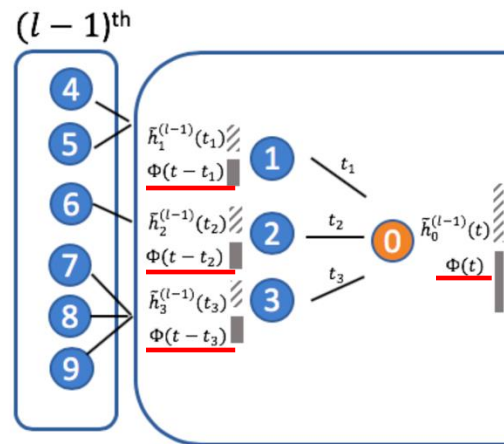
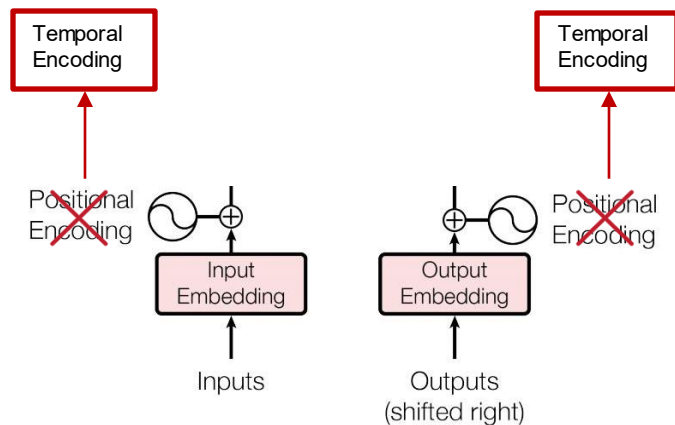
□ Importance of Temporal Feature Encoding

- Temporal information has different meanings depending on its interval of occurrence, $t_1 - t_2$
 - Latest event → higher impact
 - Older event → lower impact
- To capture different meanings, temporal features must be explicitly encoded



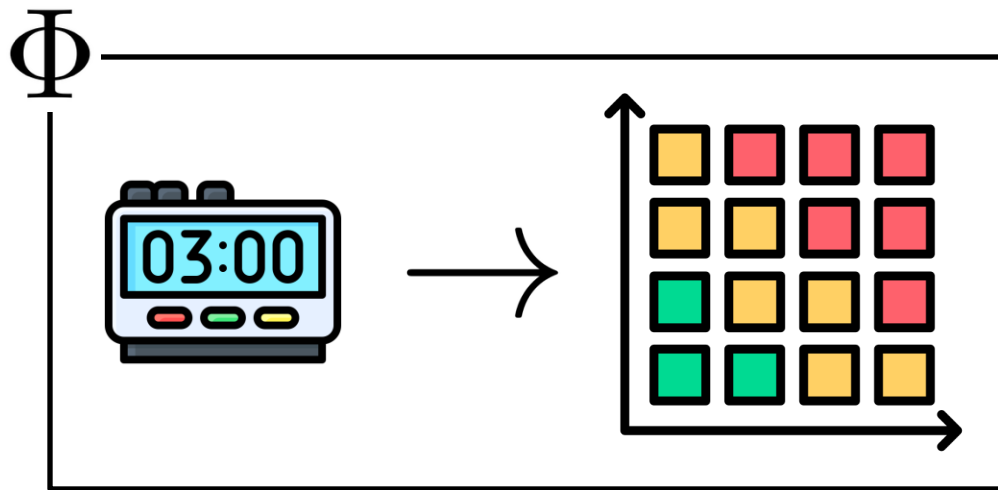
□ How to Temporal Feature Encoding

- Inspired from positional encodings of the self-attention
 - Representation of sequential information via positional encoding
- Replacement of positional encoding with temporal feature encoding



□ Functional Time Encoding

- Each timestamp is represented through a continuous functional mapping
- $\Phi : T \rightarrow \mathbb{R}^{d_T}$, $T = [0, t_{\max}]$ maps time domain to vector space
- Using Fourier transform to express a time-interval kernel in an inner-product form
 - Compatible with SAM



□ Defining Temporal Kernel

■ $\mathcal{K} : T \times T \rightarrow \mathbb{R}$ with $\mathcal{K}(t_1, t_2) := \langle \Phi(t_1), \Phi(t_2) \rangle \rightarrow$ Gram matrix

□ Naturally positive-semidefinite (PSD)

■ Translation-invariant, $\mathcal{K}(t_1 + c, t_2 + c) = \psi(t_1 - t_2) = \mathcal{K}(t_1, t_2)$ for any constant c

□ Consider only timespan, not absolute value of time

□ Bochner's Theorem

■ Continuous

■ Translation-invariant

■ Positive-semidefinite

□ Fourier Transform

$$\blacksquare \mathcal{K}(t_1, t_2) = \psi(t_1, t_2) = \int_{\mathbb{R}} e^{i\omega(t_1 - t_2)} p(\omega) d\omega = \mathbb{E}_{\omega} [\xi_{\omega}(t_1) \xi_{\omega}(t_2)^*], \quad (3)$$

$$\blacksquare \mathcal{K}(t_1, t_2) = \mathbb{E}_{\omega} [\cos(\omega(t_1 - t_2))] = \mathbb{E}_{\omega} [\cos(\omega t_1) \cos(\omega t_2) + \sin(\omega t_1) \sin(\omega t_2)]. \quad (4)$$

■ Approximating the expectation by the Monte Carlo integral,

i.e. $\mathcal{K}(t_1, t_2) := \langle \Phi(t_1), \Phi(t_2) \rangle \approx \frac{1}{d} \sum_{i=1}^d \cos(\omega_i t_1) \cos(\omega_i t_2) + \sin(\omega_i t_1) \sin(\omega_i t_2)$, with $\omega_1, \dots, \omega_d \stackrel{\text{i.i.d}}{\sim} p(\omega)$.

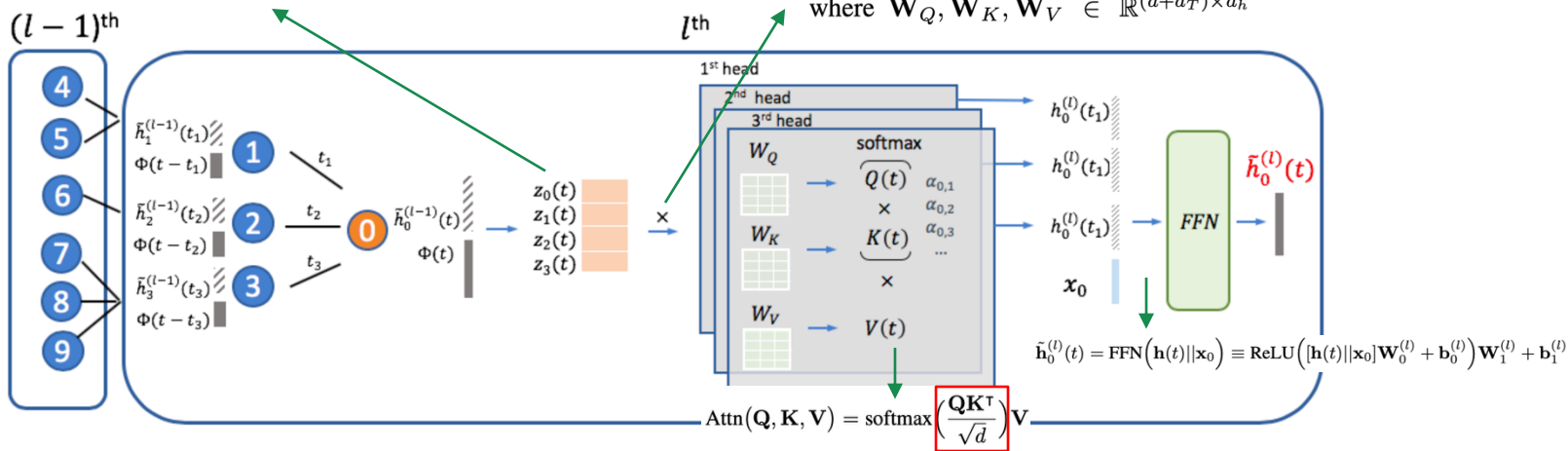
$$\underline{t \mapsto \Phi_d(t) := \sqrt{\frac{1}{d}} [\cos(\omega_1 t), \sin(\omega_1 t), \dots, \cos(\omega_d t), \sin(\omega_d t)]}, \quad (5)$$

□ Application of Temporal Encoding in Self-Attention

- Concatenation of mapped time intervals with nodes' hidden states
- Query as target node, keys/values as neighbor nodes
 - Natural timespan reflection with self-attention aggregation

$$\mathbf{Z}(t) = [\tilde{\mathbf{h}}_0^{(l-1)}(t) \parallel \Phi_{d_T}(0), \tilde{\mathbf{h}}_1^{(l-1)}(t_1) \parallel \Phi_{d_T}(t - t_1), \dots, \tilde{\mathbf{h}}_N^{(l-1)}(t_N) \parallel \Phi_{d_T}(t - t_N)]^T \quad \mathbf{q}(t) = [\mathbf{Z}(t)]_0 \mathbf{W}_Q, \mathbf{K}(t) = [\mathbf{Z}(t)]_{1:N} \mathbf{W}_K, \mathbf{V}(t) = [\mathbf{Z}(t)]_{1:N} \mathbf{W}_V$$

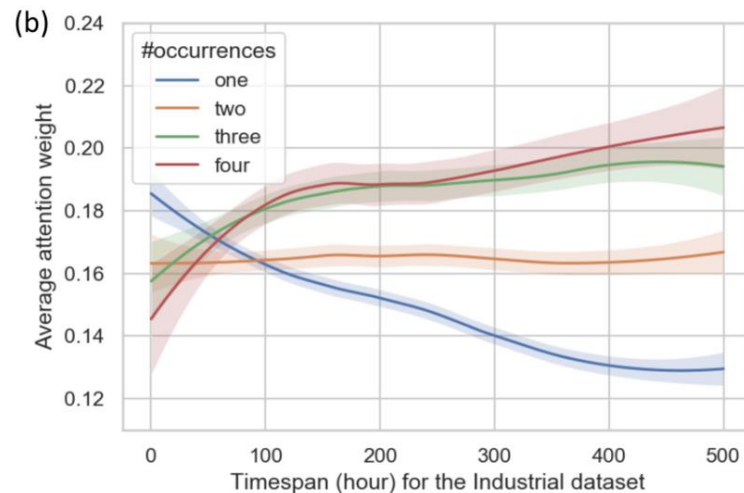
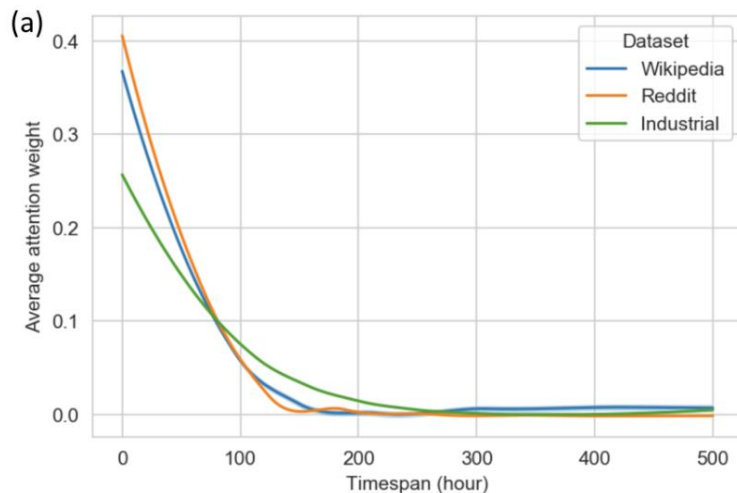
where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{(d+d_T) \times d_h}$



Experiment

□ Attention Weight Analysis

- Longer timespan $\rightarrow$ lower attention weight
- Higher frequency $\rightarrow$ higher attention weight



☐ Loss Function

- Higher similarity for connected pairs
- Lower similarity for unconnected pairs

$$\ell = \sum_{(v_i, v_j, t_{ij}) \in \mathcal{E}} -\log \left(\sigma \left(-\tilde{\mathbf{h}}_i^l(t_{ij})^\top \tilde{\mathbf{h}}_j^l(t_{ij}) \right) \right) - Q \cdot \mathbb{E}_{v_q \sim P_n(v)} \log \left(\sigma \left(\tilde{\mathbf{h}}_i^l(t_{ij})^\top \tilde{\mathbf{h}}_q^l(t_{ij}) \right) \right), \quad (8)$$

☐ Datasets

- Reddit dataset
 - ☐ Active users and their posts under subreddits
- Wikipedia dataset
 - ☐ Top edited pages and active users
- Industrial dataset
 - ☐ From Walmart website
 - ☐ The customer-product purchase

□ Transductive Learning Task Results

■ Predicting future edges of nodes

	Dataset Metric	Reddit		Wikipedia		Industrial	
		Accuracy	AP	Accuracy	AP	Accuracy	AP
Transductive	GAE	74.31 (0.5)	93.23 (0.3)	72.85 (0.7)	91.44 (0.1)	68.92 (0.3)	81.15 (0.2)
	VAGE	74.19 (0.4)	92.92 (0.2)	78.01 (0.3)	91.34 (0.3)	67.81 (0.4)	80.87 (0.3)
	DeepWalk	71.43 (0.6)	83.10 (0.5)	76.67 (0.5)	90.71 (0.6)	65.87 (0.3)	80.93 (0.2)
	Node2vec	72.53 (0.4)	84.58 (0.5)	78.09 (0.4)	91.48 (0.3)	66.64 (0.3)	81.39 (0.3)
	CTDNE	73.76 (0.5)	91.41 (0.3)	79.42 (0.4)	92.17 (0.5)	67.81 (0.3)	80.95 (0.5)
	GAT	92.14 (0.2)	97.33 (0.2)	87.34 (0.3)	94.73 (0.2)	69.58 (0.4)	81.51 (0.2)
	<u>GAT+T</u>	92.47 (0.2)	97.62 (0.2)	<u>87.57</u> (0.2)	<u>95.14</u> (0.4)	70.15 (0.3)	82.66 (0.4)
	GraphSAGE	92.31 (0.2)	97.65 (0.2)	85.93 (0.3)	93.56 (0.3)	70.19 (0.2)	83.27 (0.3)
	<u>GraphSAGE+T</u>	<u>92.58</u> (0.2)	<u>97.89</u> (0.3)	86.31 (0.3)	93.72 (0.3)	<u>71.84</u> (0.3)	<u>84.95</u> (0.)
	Const-TGAT	91.39 (0.2)	97.86 (0.2)	86.03 (0.4)	93.50 (0.3)	68.52 (0.2)	81.91 (0.3)
	TGAT	92.92 (0.3)	98.12 (0.2)	88.14 (0.2)	95.34 (0.1)	73.28 (0.2)	86.32 (0.1)

□ Inductive Learning Task Results

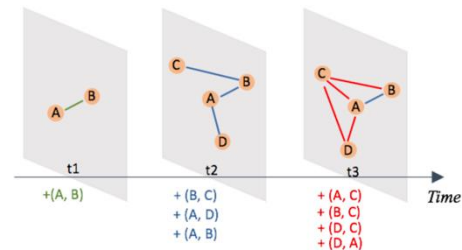
- Predicting future edges of unseen nodes

Dataset Metric	Reddit		Wikipedia		Industrial	
	Accuracy	AP	Accuracy	AP	Accuracy	AP
GAT	89.86 (0.2)	95.37 (0.3)	82.36 (0.3)	91.27 (0.4)	68.28 (0.2)	79.93 (0.3)
<u>GAT+T</u>	<u>90.44 (0.3)</u>	<u>96.31 (0.3)</u>	<u>84.82 (0.3)</u>	<u>93.57 (0.3)</u>	<u>69.51 (0.3)</u>	<u>81.68 (0.3)</u>
GraphSAGE	89.43 (0.1)	96.27 (0.2)	82.43 (0.3)	91.09 (0.3)	67.49 (0.2)	80.54 (0.3)
<u>GraphSAGE+T</u>	<u>90.07 (0.2)</u>	<u>95.83 (0.2)</u>	<u>84.03 (0.4)</u>	<u>92.37 (0.5)</u>	<u>69.66 (0.3)</u>	<u>82.74 (0.3)</u>
Const-TGAT	88.28 (0.3)	94.12 (0.2)	83.60 (0.4)	91.93 (0.3)	65.87 (0.3)	77.03 (0.4)
TGAT	90.73 (0.2)	96.62 (0.3)	85.35 (0.2)	93.99 (0.3)	72.08 (0.3)	84.99 (0.2)

Conclusion

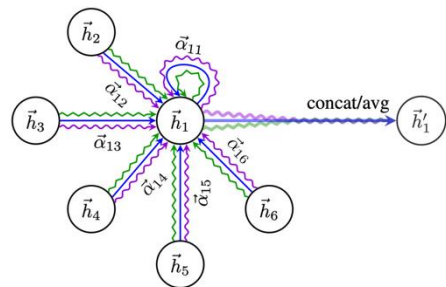
□ Temporal Factor

- Important and essential
- Challenging



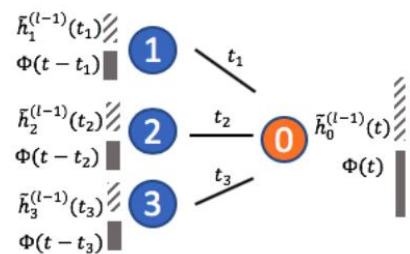
□ Inspired from GAT

- Available in the inductive setting
- Using attention mechanism between nodes



□ Functional Time Encoding with Fourier Transform

- Timestamp encoding
- Learning patterns of timespan by inner product of timestamps





Temporal Graph Networks For Deep Learning On Dynamic Graphs

Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard,
Federico Monti, Michael Brostein

Twitter

ICML 2020 workshop

Overall Process of TGN

■ Making messages from events

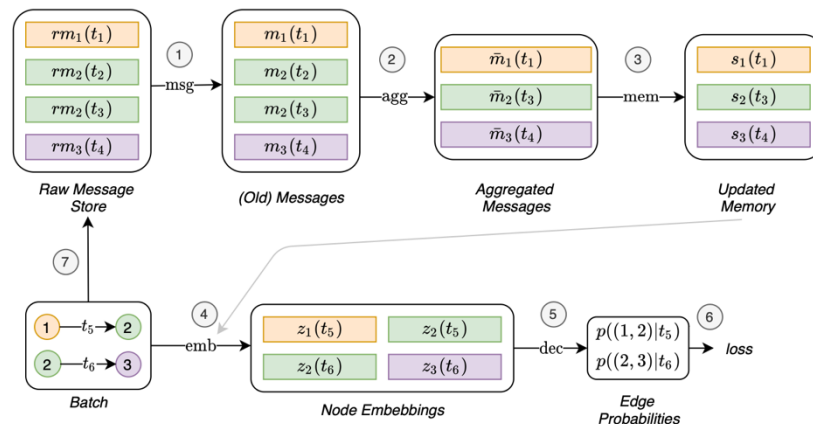
- Concatenating each features

■ Updating nodes' memory using new messages

- Using GRU

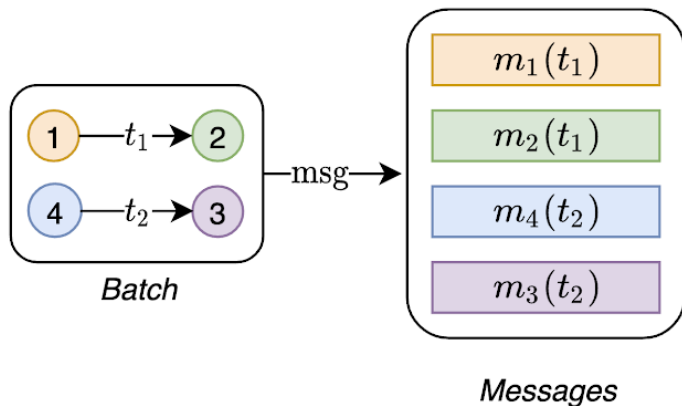
■ Embedding nodes by aggregating their memory with its neighbors

- Using SAM



□ Message Functions on TGN

- Computing messages for the event
- Aggregation of multiple messages into a single one
- The messages are used to update the memory



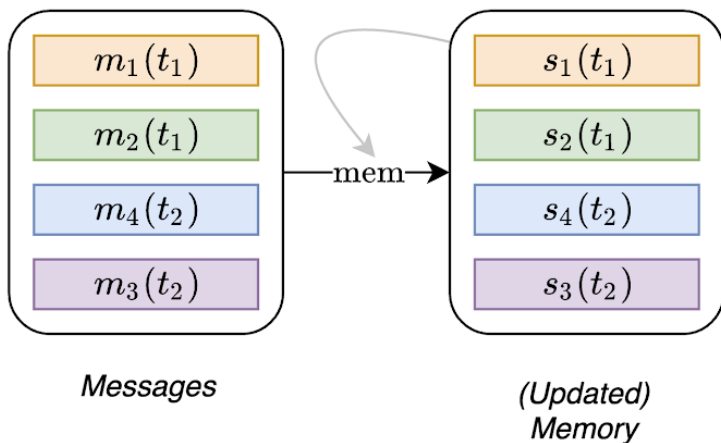
$$\mathbf{m}_i(t) = \text{msg}(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), t, \mathbf{e}_{ij}(t))$$

$$\mathbf{m}_j(t) = \text{msg}(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), t, \mathbf{e}_{ij}(t))$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

□ Memory Modules on TGN

- Storing the states of all the nodes
- Representing the node's history in a compressed format
- Updated after an event with the new messages

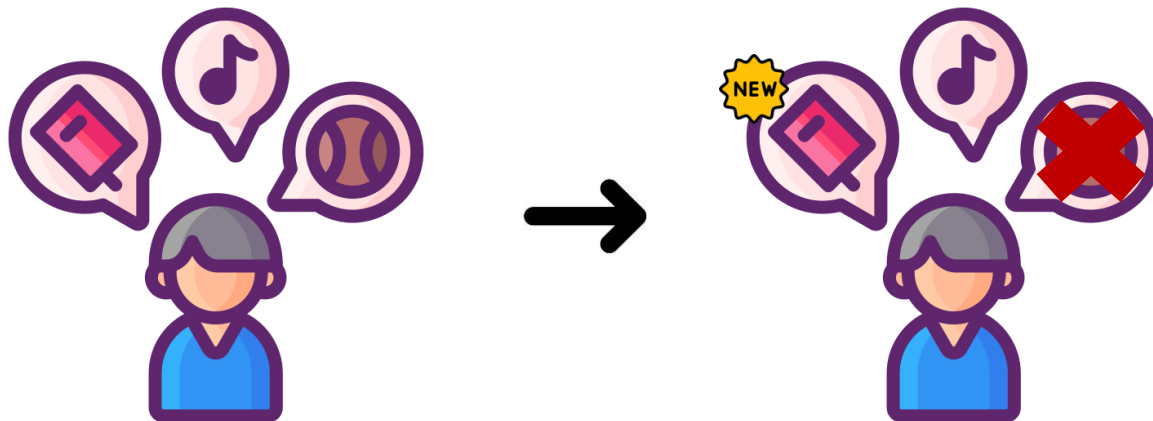


$$\mathbf{s}_i(t) = \text{mem}(\mathbf{m}_i(t), \mathbf{s}_i(t^-))$$

TGN - Node Embedding

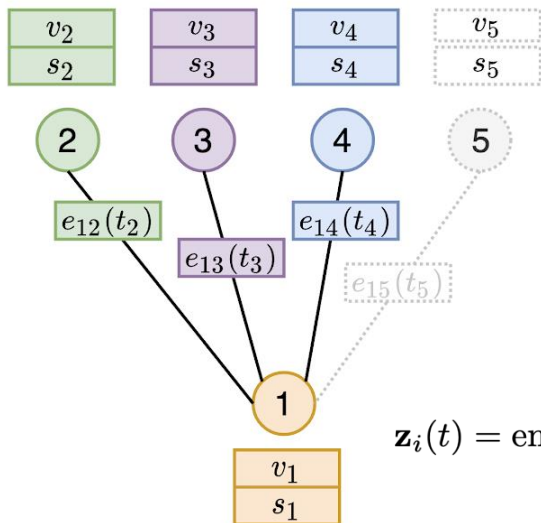
☐ Directly Embedding Using Node's Memory

- A bad idea due to the **staleness problem**
- The node with long period of inactivity, its memory goes out of date
 - ☐ Imagine returning to social media after a long break



□ Aggregation to Solve the Staleness Problem

- Some of neighbors have been active
- By aggregating their memories, TGN can compute up-to-date embeddings



$$\mathbf{h}_i^{(l)}(t) = \text{MLP}^{(l)}(\mathbf{h}_i^{(l-1)}(t) \parallel \tilde{\mathbf{h}}_i^{(l)}(t)),$$

$$\tilde{\mathbf{h}}_i^{(l)}(t) = \text{MultiHeadAttention}^{(l)}(\mathbf{q}^{(l)}(t), \mathbf{K}^{(l)}(t), \mathbf{V}^{(l)}(t)),$$

$$\mathbf{q}^{(l)}(t) = \mathbf{h}_i^{(l-1)}(t) \parallel \phi(0),$$

$$\mathbf{K}^{(l)}(t) = \mathbf{V}^{(l)}(t) = \mathbf{C}^{(l)}(t),$$

$$\mathbf{C}^{(l)}(t) = [\mathbf{h}_1^{(l-1)}(t) \parallel \mathbf{e}_{i1}(t_1) \parallel \phi(t - t_1), \dots, \mathbf{h}_N^{(l-1)}(t) \parallel \mathbf{e}_{iN}(t_N) \parallel \phi(t - t_N)].$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t))$$

□ A Comparison of Various GNNs

■ Future link prediction task in transductive and inductive settings

	Wikipedia		Reddit		Twitter	
	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive
GAE*	91.44 ± 0.1	†	93.23 ± 0.3	†	—	†
VAGE*	91.34 ± 0.3	†	92.92 ± 0.2	†	—	†
DeepWalk*	90.71 ± 0.6	†	83.10 ± 0.5	†	—	†
Node2Vec*	91.48 ± 0.3	†	84.58 ± 0.5	†	—	†
GAT*	94.73 ± 0.2	91.27 ± 0.4	97.33 ± 0.2	95.37 ± 0.3	67.57 ± 0.4	62.32 ± 0.5
GraphSAGE*	93.56 ± 0.3	91.09 ± 0.3	97.65 ± 0.2	96.27 ± 0.2	65.79 ± 0.6	60.13 ± 0.6
CTDNE	92.17 ± 0.5	†	91.41 ± 0.3	†	—	†
Jodie	94.62 ± 0.5	93.11 ± 0.4	97.11 ± 0.3	94.36 ± 1.1	85.20 ± 2.4	79.83 ± 2.5
TGAT	95.34 ± 0.1	93.99 ± 0.3	98.12 ± 0.2	96.62 ± 0.3	70.02 ± 0.6	66.35 ± 0.8
DyRep	94.59 ± 0.2	92.05 ± 0.3	97.98 ± 0.1	95.68 ± 0.2	83.52 ± 3.0	78.38 ± 4.0
TGN-attn	98.46 ± 0.1	97.81 ± 0.1	98.70 ± 0.1	97.55 ± 0.1	94.52 ± 0.5	91.37 ± 1.1

Experiment

□ A Comparison of Various GNNs

■ Dynamic node classification on the same settings

	Wikipedia	Reddit
GAE*	74.85 ± 0.6	58.39 ± 0.5
VAGE*	73.67 ± 0.8	57.98 ± 0.6
GAT*	82.34 ± 0.8	64.52 ± 0.5
GraphSAGE*	82.42 ± 0.7	61.24 ± 0.6
CTDNE	75.89 ± 0.5	59.43 ± 0.6
JODIE	84.84 ± 1.2	61.83 ± 2.7
TGAT	83.69 ± 0.7	65.56 ± 0.7
DyRep	84.59 ± 2.2	62.91 ± 2.4
TGN-attn	87.81 ± 0.3	67.06 ± 0.9

Conclusion

□ Using RNN on Each of Nodes with Messages

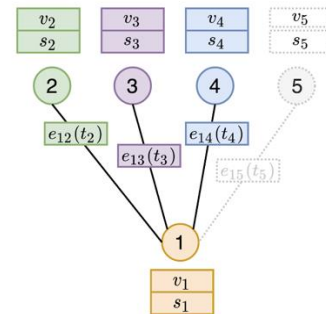
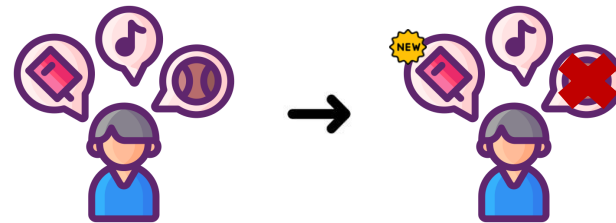
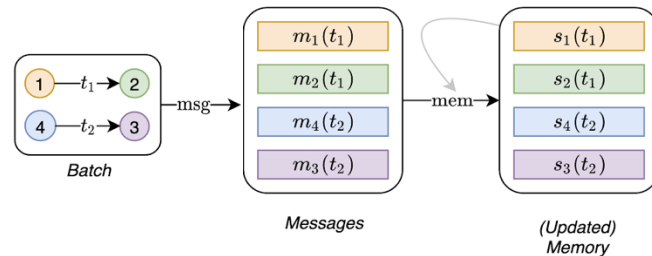
- Similar to JODIE

□ Directly Embedding Using Node's Memory

- Enable to cause a staleness problem

□ Aggregating a memory of neighbors

- Using SAM with a time encoding
- Similar to TGAT





Do We Really Need Complicated Model Architecture For Temporal Networks?

Weilin Cong¹, Si Zhang², Jian Kang³, Baichuan Yuan², Hao Wu², Xin Zhou²,
Hanghang Tong³, Mehrdad Mahdavi¹

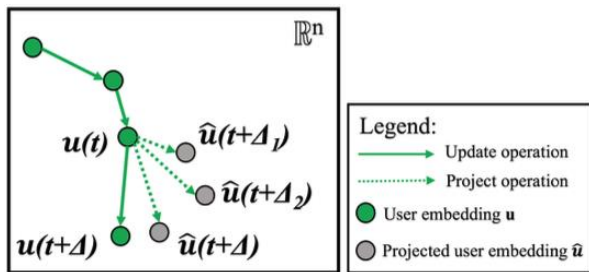
1. Penn State, 2. Meta, 3. University of Illinois at Urbana-Champaign

ICLR 2023

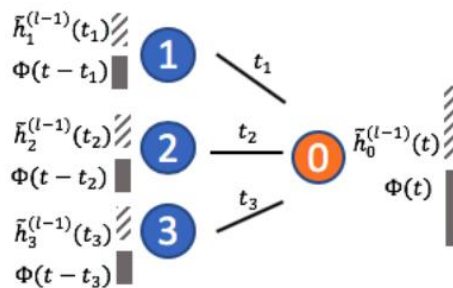
Background

□ Various Ways of Temporal Graph Learning

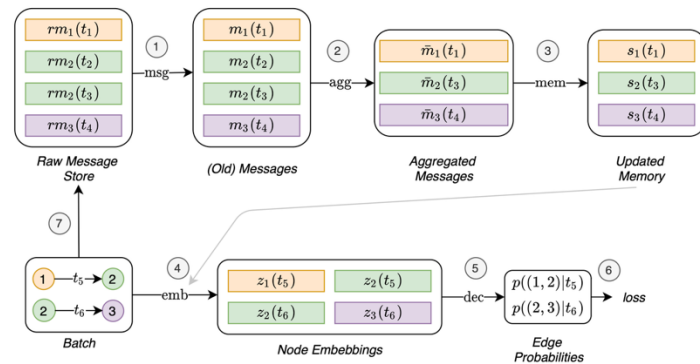
- Recognized as an important machine learning problem
- A wealth of high-impact applications
- Used for temporal link prediction
 - Future interactions among nodes



JODIE



TGAT

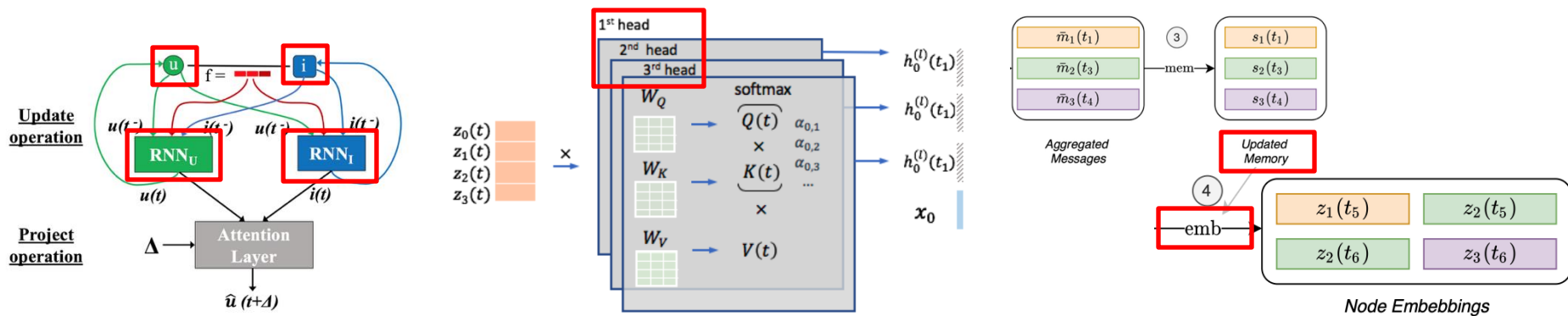


TGN

Background

□ Using RNN or Self-Attention Mechanism

- JODIE: Inputs $\rightarrow$ RNN + memory blocks
- TGAT: Inputs $\rightarrow$ self-attention mechanism with time-encoding
- TGN: Inputs $\rightarrow$ RNN + memory blocks $\rightarrow$ self-attention mechanism with time-encoding



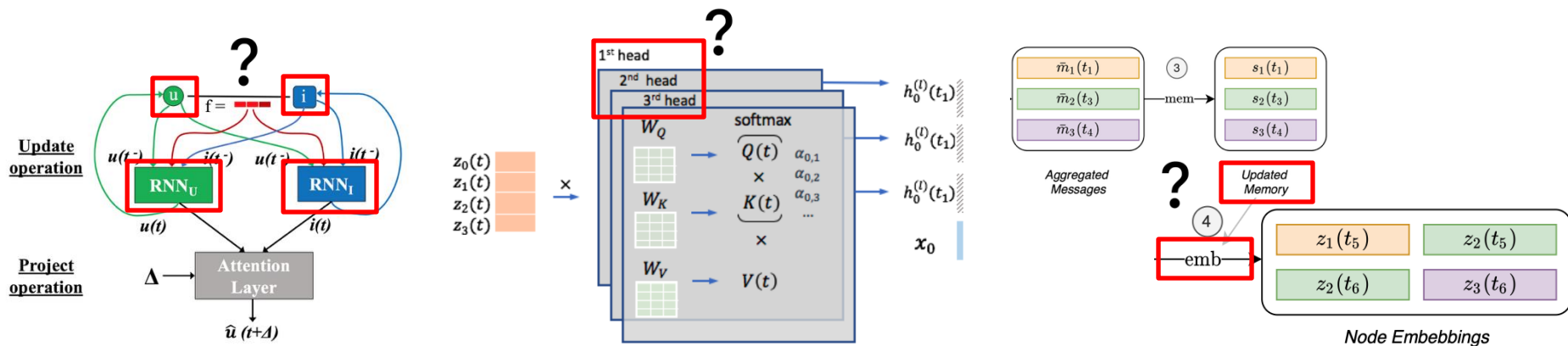
Background

□ Although Powerful Performance

■ Conceptually and technically complicated

- Which parts of the model design truly contribute to its success
- Whether these components are indispensable

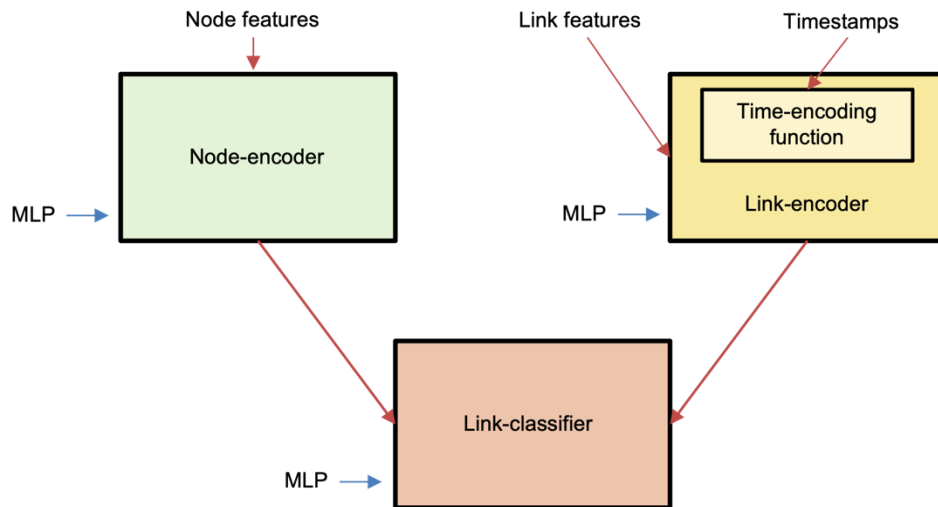
■ Questioning the indispensability of RNN and SAM



□ GraphMixer

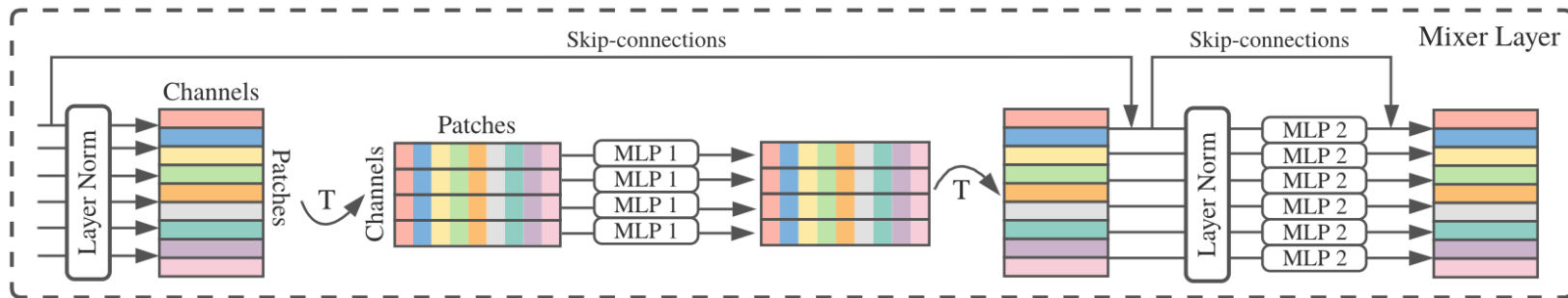
■ A simple architecture for temporal graph learning to predict future links

- Multi-layer perceptrons
- A fixed time-encoding function
- Obtaining outstanding results and generalizing better



□ MLP-Mixer

- An all-MLP architecture for vision
- Token-mixing
- Channel-mixing



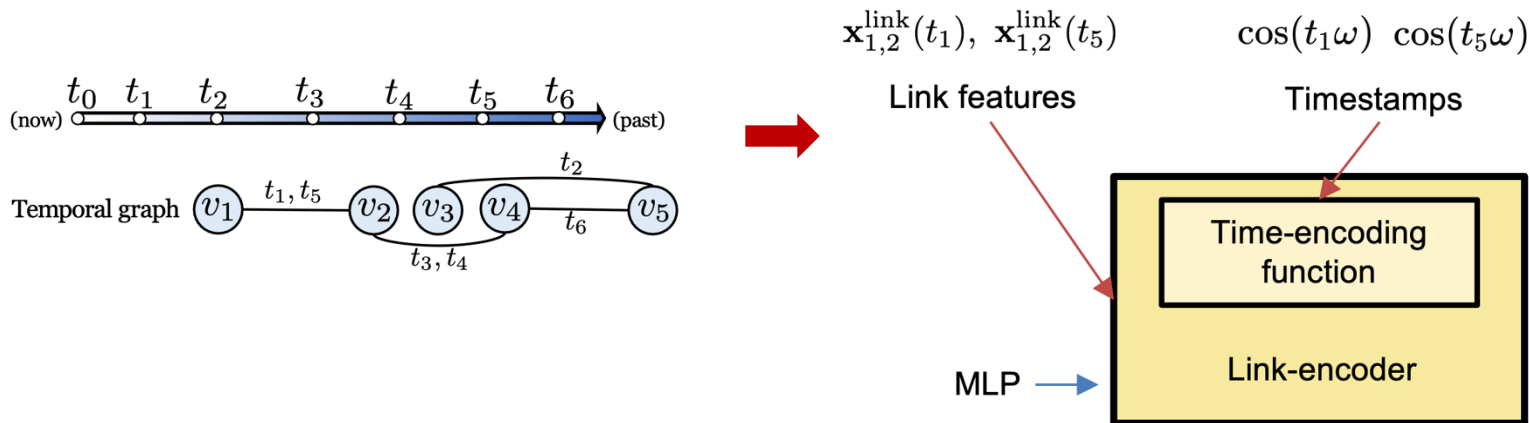
□ A Link-Encoder

■ Summarizing the information from temporal links

- Link features and link timestamps

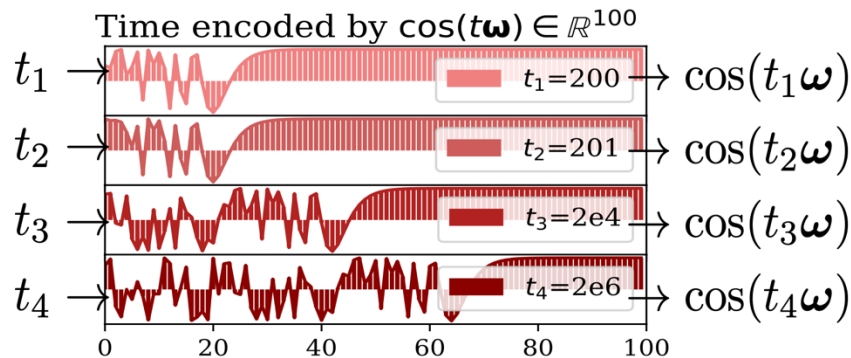
■ Need to distinguish different timestamps and temporal link information

- A time-encoding function



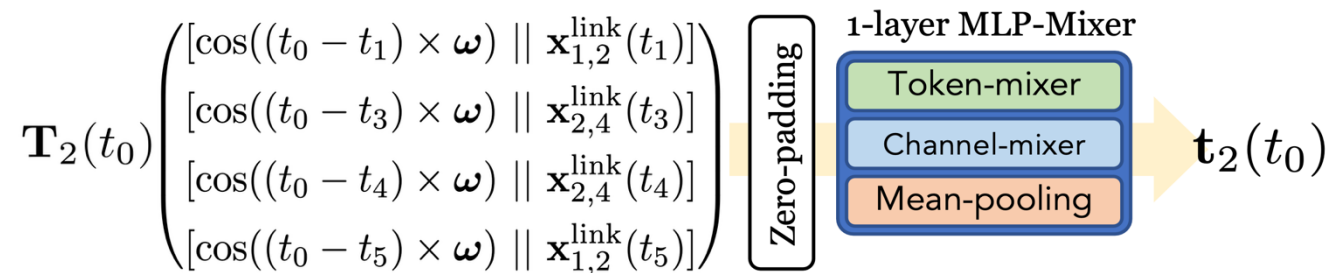
□ A Time-Encoding Function

- Using $\cos(t\omega)$, utilizing features $\omega = \{\alpha^{-(i-1)}/\beta\}_{i=1 \rightarrow d}$
 - Encoding each timestamps into a d-dimensional vector **with fixed ω**
- Distinguishing different timestamps
 - Similar timestamps $\rightarrow$ similar time-encodings
 - Larger timestamps $\rightarrow$ convergence slowly
- Concatenation encodings with their corresponding link features
 - We want to predict $t_0, (t_1, \mathbf{x}_{1,2}^{\text{link}}(t_1))$ as $[\cos((t_0 - t_1)\omega) \parallel \mathbf{x}_{1,2}^{\text{link}}(t_1)]$



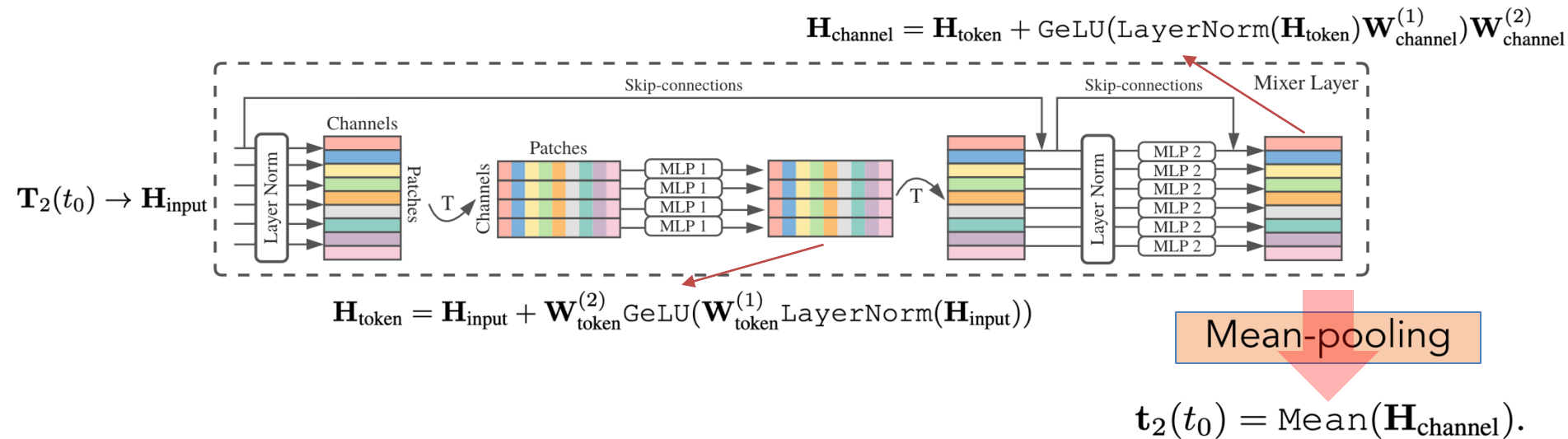
□ Mixer for Information Summarizing

- Stacking all links' information into a big matrix
- Zero-padding to the fixed length K
 - To capture how often a node interacts with other nodes
- Using a 1-layer MLP-mixer



□ MLP-Mixer with Temporal Links

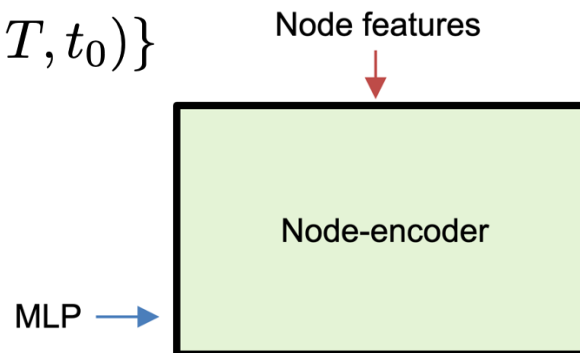
- A good alternative to self attention
- Token-mixing → Captures global structure across links
- Channel-mixing → Mixes features capturing overall structure



□ A Node-Encoder

- Capturing the node identity and node feature information
 - Via neighbor mean-pooling
- Computing only based on the 1-hop neighbors, most recent neighbors
- Using one-hot node representations for dataset without node features

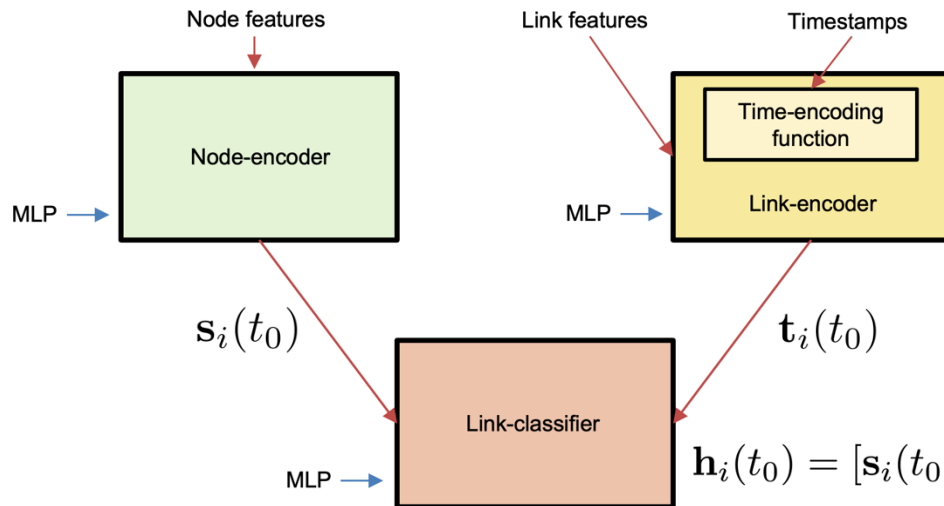
$$\mathbf{s}_i(t_0) = \mathbf{x}_i^{\text{node}} + \text{Mean}\{\mathbf{x}_j^{\text{node}} \mid v_j \in \mathcal{N}(v_i; t_0 - T, t_0)\}$$



□ A Link-Classifier

■ Classifying whether a link exists at time t_0

- Using the output of link-encoder and the output of node-encoder



between node v_i, v_j
 $[\mathbf{h}_i(t_0) \parallel \mathbf{h}_j(t_0)]$, i.e., $p_{ij} = \text{MLP}([\mathbf{h}_i(t_0) \parallel \mathbf{h}_j(t_0)])$

$$\mathbf{h}_i(t_0) = [\mathbf{s}_i(t_0) \parallel \mathbf{t}_i(t_0)]$$

□ Average Precision Score

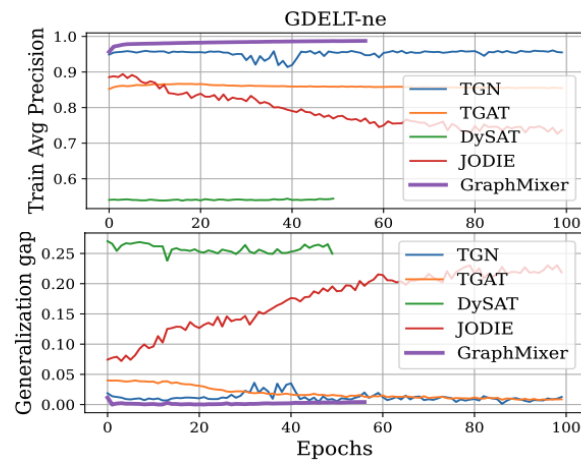
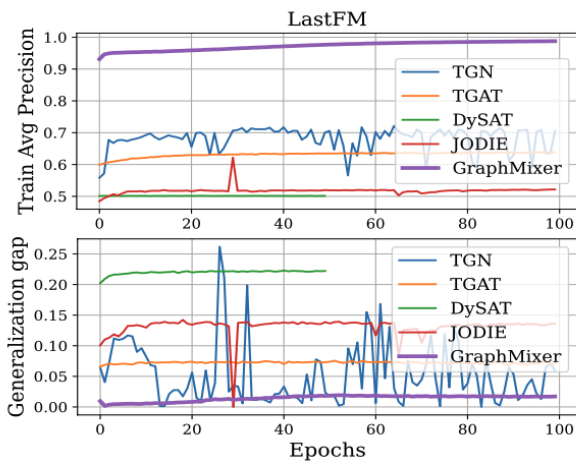
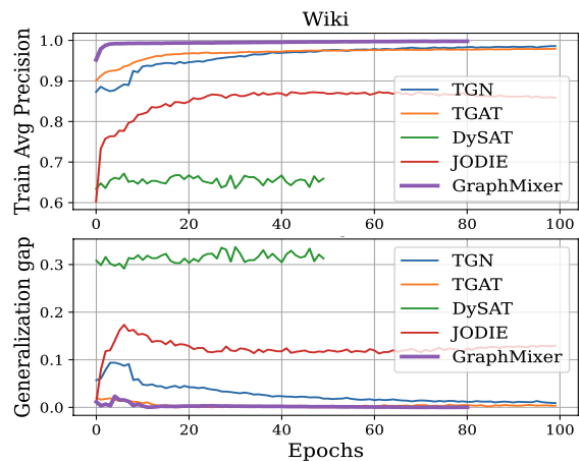
- GraphMixer outperforms all baselines on all datasets
- Successful fixed time-encoding function

	Reddit L, T	Wiki L, T	MOOC T	LastFM T	GDELTA L, N, T	GDELTA-ne T	GDELTA-e N, T
JODIE	99.30 ± 0.01	98.81 ± 0.01	99.16 ± 0.01	67.51 ± 0.87	98.27 ± 0.02	97.13 ± 0.02	96.96 ± 0.02
DySAT	98.52 ± 0.01	96.71 ± 0.02	98.82 ± 0.02	76.40 ± 0.77	98.52 ± 0.02	82.47 ± 0.13	97.25 ± 0.02
TGAT	99.66 ± 0.01	97.75 ± 0.02	98.43 ± 0.01	54.77 ± 1.01	98.25 ± 0.02	84.30 ± 0.10	96.96 ± 0.02
TGN	99.80 ± 0.01	99.55 ± 0.01	99.62 ± 0.01	82.23 ± 0.50	98.15 ± 0.02	97.13 ± 0.02	96.04 ± 0.02
CAWs-mean	98.43 ± 0.02	97.72 ± 0.03	62.99 ± 0.87	76.35 ± 0.08	95.11 ± 0.12	69.20 ± 0.10	91.72 ± 0.19
CAWs-attn	98.51 ± 0.02	97.95 ± 0.03	63.07 ± 0.82	76.31 ± 0.10	95.06 ± 0.11	69.54 ± 0.19	91.54 ± 0.22
TGSRec	95.21 ± 0.08	91.64 ± 0.12	83.62 ± 0.34	76.91 ± 0.87	97.03 ± 0.61	97.03 ± 0.61	97.03 ± 0.61
APAN	99.24 ± 0.02	98.14 ± 0.01	98.70 ± 0.98	69.39 ± 0.81	95.96 ± 0.10	97.38 ± 0.23	96.77 ± 0.18
GraphMixer-L	99.84 ± 0.01	99.70 ± 0.01	99.81 ± 0.01	95.50 ± 0.03	98.99 ± 0.02	96.14 ± 0.02	98.99 ± 0.02
GraphMixer-N	99.24 ± 0.01 ^h	90.33 ± 0.01 ^h	97.35 ± 0.02 ^h	63.80 ± 0.03 ^h	94.44 ± 0.02	96.00 ± 0.02 ^h	98.81 ± 0.02 ^h
GraphMixer	99.93 ± 0.01 ^h	99.85 ± 0.01 ^h	99.91 ± 0.01 ^h	96.31 ± 0.02 ^h	98.89 ± 0.02	98.39 ± 0.02 ^h	98.22 ± 0.02 ^h

Experiment

□ Training Set Average Precision and Generalization Gap

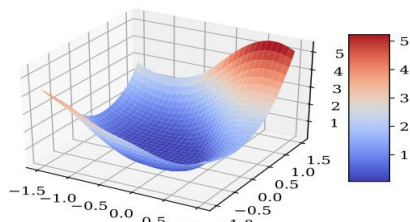
■ Better convergence and generalization ability with a GraphMixer



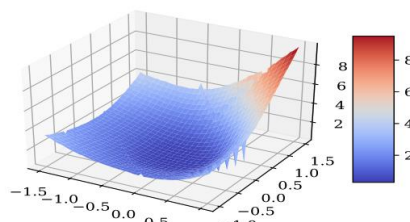
Experiment

□ Training Loss Landscape

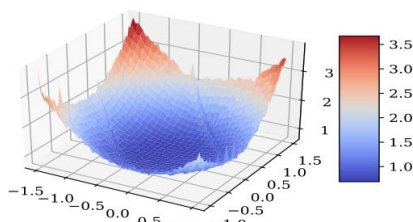
■ Smoother loss landscapes with a GraphMixer



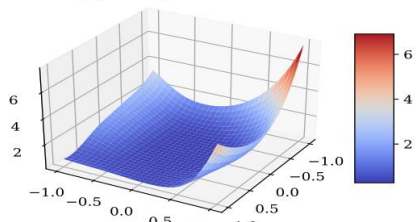
(a) GraphMixer on Wiki



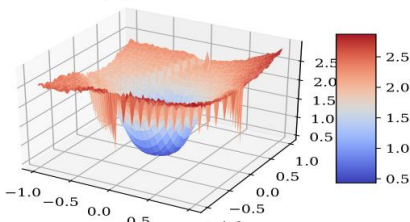
(b) TGAT on Wiki



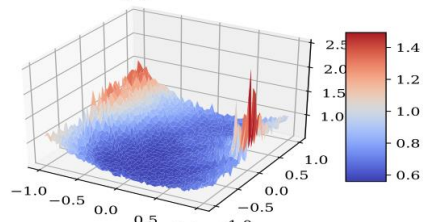
(c) TGN on Wiki



(d) GraphMixer on GDELT-e



(e) TGAT on GDELT-e



(f) TGN on GDELT-e

Experiment

□ Average Precision Score with Fixed/Trainable Time Encoding Function

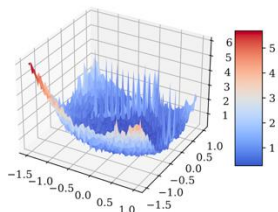
■ Better model performance with fixed time encoding function

	Reddit	Wiki	MOOC
JODIE	99.30 → 99.76	98.81 → 99.00	99.16 → 99.17
TGAT	98.66 → 99.48	96.71 → 98.55	98.43 → 99.33
TGN	99.80 → 99.83	99.55 → 99.54	99.62 → 99.62
	LastFM	GDELT-ne	GDELT-e
	67.51 → 79.89	97.13 → 98.23	96.96 → 96.96
	54.77 → 76.26	84.30 → 92.31	96.96 → 96.28
	82.23 → 87.58	98.15 → 98.25	96.04 → 97.34

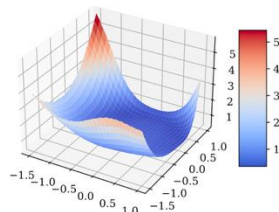
Existing Methods with Fixed Time-Encoding Function

■ Smoother optimization landscape using only fixed time-encoding

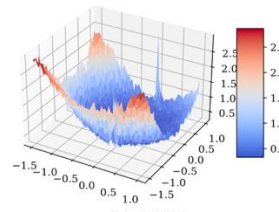
- Leading a faster convergence speed
- A better generalization



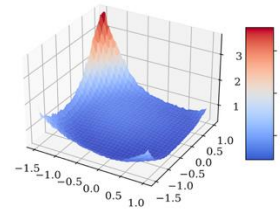
(d) JODIE



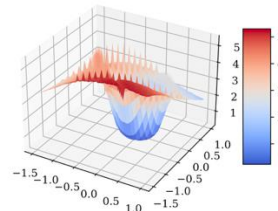
(c) MOOC



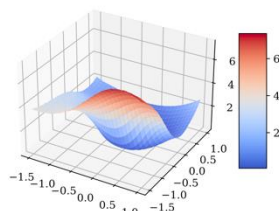
(a) TGN



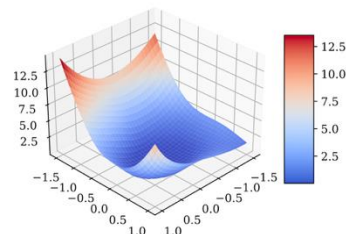
(c) MOOC



(b) TGAT



(c) MOOC



(e) GraphMixer

□ Average Precision Score of GraphMixer with Different Input Data

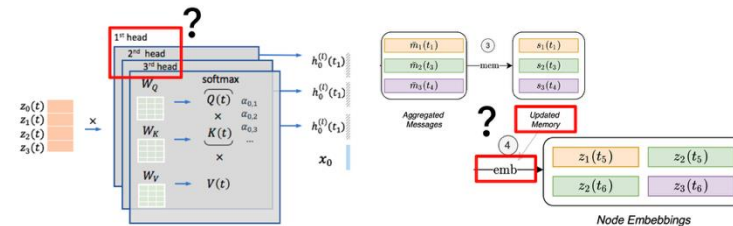
- Undirected temporal graph
- Fixed relative time-encoding
- 1-hop neighbors, most recent neighbors

			Reddit	Wiki	MOOC	LastFM
Direct vs undirect temporal graph		Directed temporal graph	99.69	88.37	97.87	78.34
		Undirected temporal graph	99.93	99.85	99.91	96.31
Time information		Relative timestamp $(t_i - t_0)$	99.79	99.80	99.81	95.32
		Relative time-encoding $\cos((t_i - t_0)\omega)$	99.93	99.85	99.91	96.31
		Absolute timestamp t_i	98.90	98.23	98.73	92.25
		Absolute time-encoding $\cos(t_i\omega)$	99.52	99.13	99.74	95.28
Neighbor selection	2-hop	Most recent neighbors	99.39	98.05	99.11	89.36
	1-hop	Most recent neighbors	99.93	99.85	99.91	96.31
	2-hop	Uniform sample neighbors	97.66	92.57	98.87	65.72
	1-hop	Uniform sample neighbors	98.19	94.74	98.40	60.02

Conclusion

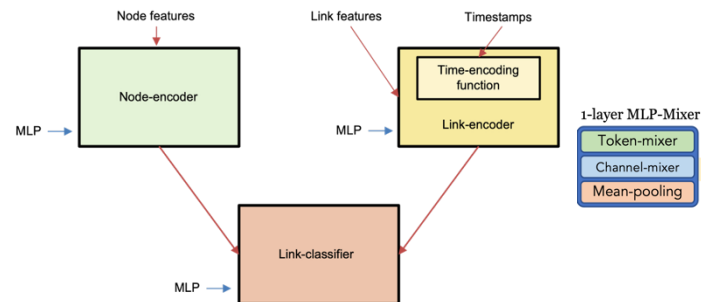
Existing Methods

- Using RNN, SAM
- Non-trivial to understand the model



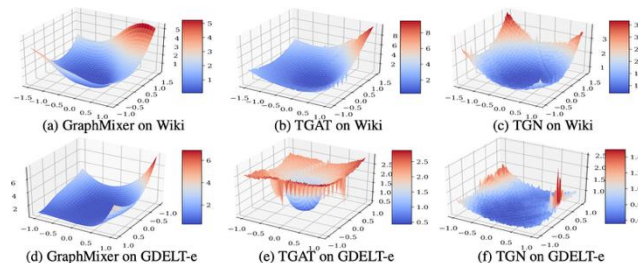
GraphMixer

- An MLP-mixer
- A Fixed time-encoding function
- Stable and fast convergence



Various Extensive Experiments

- Outstanding performance despite only use MLPs



□ t-Batch for Parallel Training for JODIE

- Independence within a batch ensuring no duplicate users or items
- Temporal consistency across batches guaranteeing sequential order of embeddings

$$\text{batch}(S_{r+1}) = 1 + \max(\text{maxBatch}(u, r), \text{maxBatch}(i, r))$$

Batch 1 = {(1)}

Batch 2 = {(2, 3)}

Batch 3 = {(4, 5)}

1. (u_1, i_1, t_1)
2. (u_1, i_2, t_2)
3. (u_2, i_1, t_3)
4. (u_2, i_3, t_4)
5. (u_1, i_1, t_5)

□ Explanations of Various Configure of TGN and Existing Methods

	Mem.	Mem. Updater	Embedding	Mess. Agg.	Mess. Func.
Jodie	node	RNN	time	— [†]	id
TGAT	—	—	attn (2l, 20n)*	—	—
DyRep	node	RNN	id	— [‡]	attn
TGN-attn	node	GRU	attn (1l, 10n)	last	id
TGN-2l	node	GRU	attn (2l, 10n)	last	id
TGN-no-mem	—	—	attn (1l, 10n)	—	—
TGN-time	node	GRU	time	last	id
TGN-id	node	GRU	id	last	id
TGN-sum	node	GRU	sum (1l, 10n)	last	id
TGN-mean	node	GRU	attn (1l, 10n)	mean	id